

Week3_예습과제_이재린

태그

예습과제

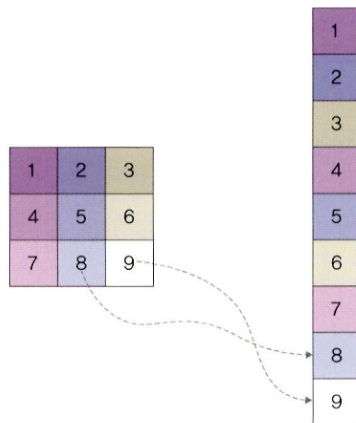
CH5. 합성곱 신경망 1

▼ 1. 합성곱 신경망

: 이미지 전체를 한 번에 계산하는 것이 아닌 이미지의 국소적 부분을 계산 > 시간과 자원 절약하며 이미지의 세밀한 부분까지 분석 가능

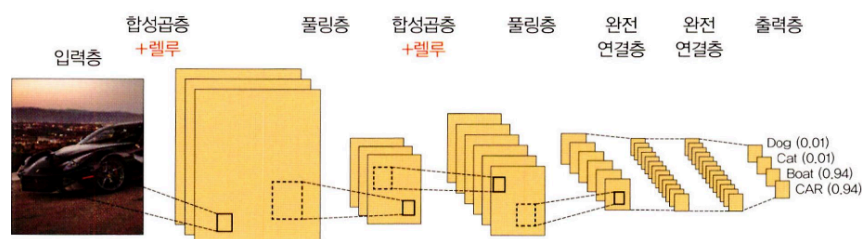
합성곱층의 필요성

- 이미지 분석의 flow : 2차원 배열을 펼쳐서 각 픽셀에 가중치 곱하여 은닉층으로 전달 >> 데이터의 공간적 구조가 무시됨
- 이를 방지하는게 합성곱층



합성곱 신경망 구조

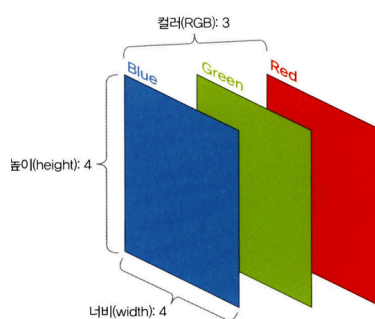
- 합성곱 신경망(CNN)
 - 음성 인식, 이미지/영상 인식에 주로 사용되는 신경망 >> 다차원 배열 처리에 특화
 - 입력층+합성곱층+풀링층+완전연결층+풀러층 으로 구성



- flow : 합성곱층&풀링층을 거치면서 이미지의 주요 특성 벡터 추출 > 완전 연결층을 거치며 1차원 벡터로 변환 > 출력층에서 활성화 함수 softmax를 사용하여 결과 출력

1. 입력층

- 이미지 : 높이 & 너비 & 채널의 값을 갖는 3차원 데이터
 - 채널 : 이미지가 gray scale일때 1, RGB면 3



2. 합성곱층

- 합성곱층 : 입력 데이터에서 특성을 추출하는 역할 수행 > 이미지에 대한 특성을 감지하기 위해 커널이나 필터를 사용 > 특성 맵을 추출

- 스트라이드 : 필터를 적용하는 위치의 간격

[gray scale]

(1) 입력 이미지에 3X3 필터 적용 : 대응되는 숫자끼리 곱한 후 모두 더해서 출력층



(2) 필터가 1만큼 이동..



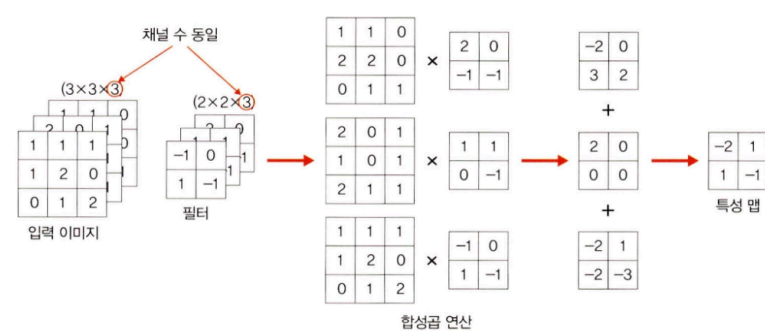
...계속 1만큼 이동



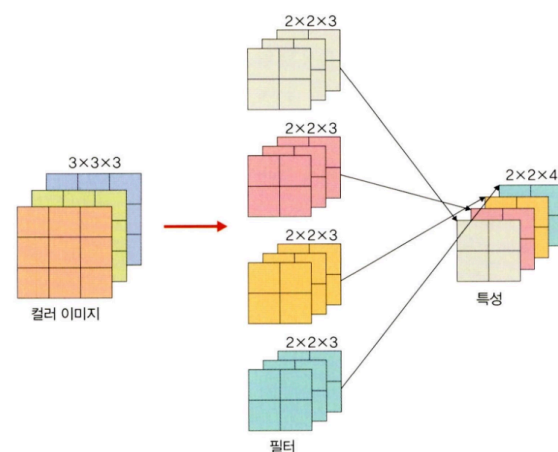
: 원본 (6,6,1) 크기가 (4,4,1) 크기의 특성 맵으로 줄어든다

[RGB : 필터 채널 3개]

: RGB 각각에 서로 다른 가중치로 합성곱을 적용 but 필터 '채널'이 3개지 필터는 1개



: 필터가 2 이상인 합성



합성곱층에 대한 요약

- 입력 데이터: $W_1 \times H_1 \times D_1$ (W_1 : 가로, H_1 : 세로, D_1 : 채널 또는 깊이)
- 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P
- 출력 데이터
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$
 - $D_2 = K$

3. 풀링층

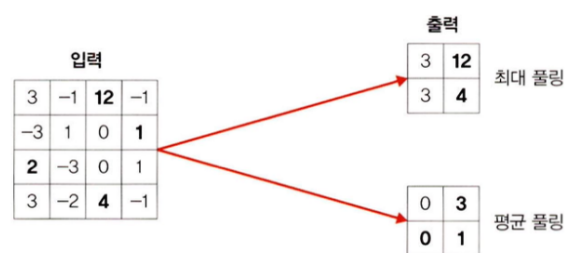
- 풀링층 : 특성 맵의 차원을 다운 샘플링하여 연산을 감소시키고, 주요 특성 벡터를 추출
- 풀링 연산
 - 최대 풀링 : 대상 영역에서 최대값 추출 >> 대부분



↳ 2칸씩 움직이며 최대값을 추출한다.

- 평균 풀링 : 대상 영역에서 평균을 반환 >> 각 커널 값을 평균화 시켜 가중치의 특성이 희미해짐

[최대 풀링과 평균 풀링]

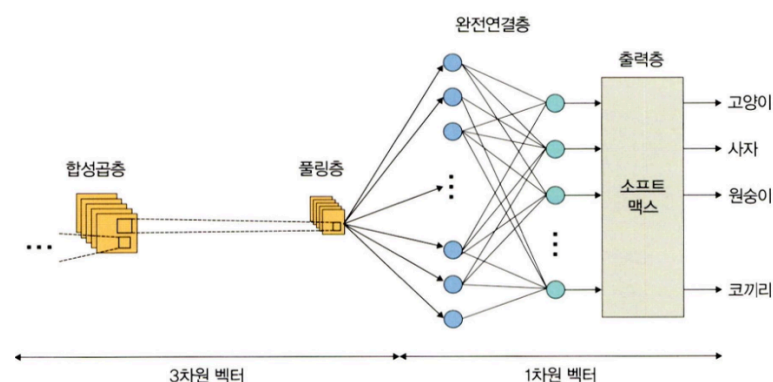


풀링층에 대한 요약

- 입력 데이터: $W_1 \times H_1 \times D_1$
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

4. 완전연결층

- 완전연결층 : 합성곱층 + 풀링층을 거치면서 특성 맵의 차원은 축소되어 완전 연결층으로 전달 >> 이미지는 3차원에서 1차원 벡터로 펼쳐짐



5. 출력층

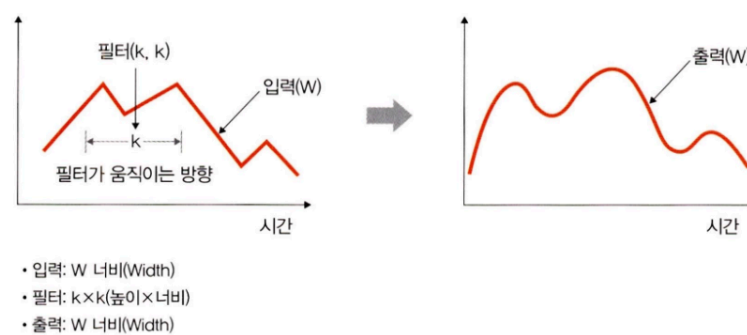
- 소프트맥스 활성화 함수가 활용됨 > 입력받은 값을 0~1 사이의 값으로 출력 > 이미지가 각 레이블에 속할 확률 값이 출력되며, 가장 높은 확률 값이 레이블의 최종 값으로 선정됨.

1D, 2D, 3D 합성곱

: 합성곱의 이동 방향과 출력 형태에 따라 다음과 같이 분류

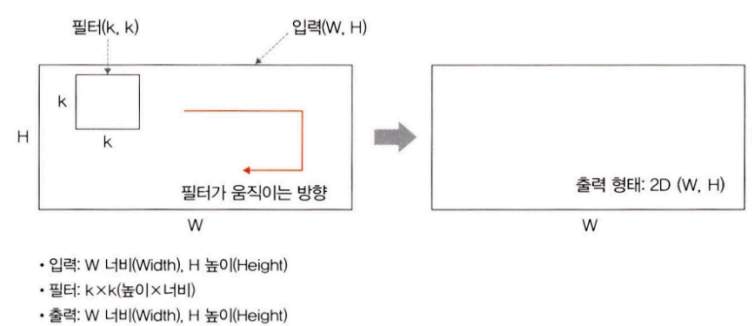
1. 1D 합성곱

- 필터가 시간을 축으로 좌우로만 이동하는 합성곱 >> 입력(W)와 필터(k)에 대한 출력은 W가 됨.
- 출력 형태 >> 1D 배열 / 그래프 곡선을 완화할 때 사용됨



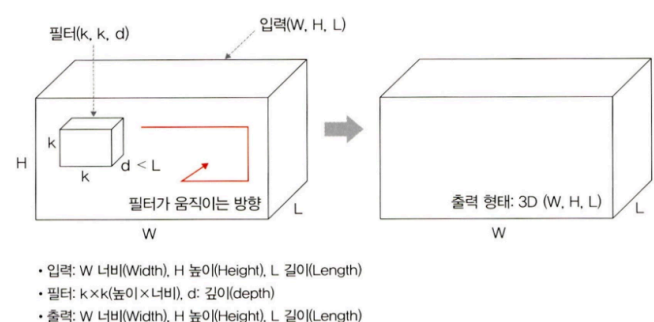
2. 2D 합성곱

- 방향 두 개로 움직이는 형태 >> 입력(W,H)과 필터(k,k)에 대한 출력은 (W,H)
- 출력 형태 >> 2D 배열



3. 3D 합성곱

- 방향 세 개로 움직이는 형태 >> 입력(W,H,L)과 필터(k,k,d)에 대한 출력은 (W,H,L)
- 출력 형태 >> 3D 배열 / d < L을 유지



4. 3D 입력을 갖는 2D 합성곱

- 입력이 3D임에도 출력 형태가 2D 행렬을 취하는 것 >> 이때 '입력 채널의 길이 L == 필터에 대한 길이 L'
- 입력(W,H,L)에 필터(k,k,L)를 적용하여 출력(W,H)을 생성
- 예시 : LeNet-5, VGG

5. 1x1 합성곱

- 3D 형태로 입력됨 >> 입력(W,H,L)에 필터(1,1,L)를 적용하여 출력(W,H) >> 연산량 감소 효과
- 예시 : GoogLeNet

▼ 2. 합성곱 신경망 맛보기

[fashion_mnist 데이터셋을 사용]

: 운동화, 샌들, 셔츠 같은 작은 이미지의 모음 / 열 가지(레이블)로 분류될 수 있는 28*28 픽셀의 이미지 7만개(훈련 데이터, 넘파이 배열)로 구성

1. 라이브러리 호출

```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.autograd import Variable
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms #데이터 전처리를 위해 사용되는 라이브러리
from torch.utils.data import Dataset, DataLoader
```

2. CPU 혹은 GPU 장치 확인

```
device=torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

3. fashion_mnist 데이터셋 내려받기

```
#fashion_mnist 데이터셋 내려받기
train_dataset=torchvision.datasets.FashionMNIST("/Users/bluecloud/Documents/대학/유런",download=True,transform=
test_dataset=torchvision.datasets.FashionMNIST("/Users/bluecloud/Documents/대학/유런",download=True,train=False
```

- 첫 번째 파라미터 : 내려받을 위치
- download: True로 설정하면 첫 번째 파라미터의 위치에 해당 데이터셋이 있는지 확인 후 내려받기
- transform : 이미지를 텐서(0~1)로 변경

4. fashion_mnist 데이터를 메모리로 불러오기 위해 DataLoader에 전달

```
train_loader=torch.utils.data.DataLoader(train_dataset,batch_size=100)
test_loader=torch.utils.data.DataLoader(test_dataset,batch_size=100)
```

5. 데이터 살펴보기 : 20개의 이미지를 레이블 정보와 함께 출력

```
# 분류에 사용될 클래스 정의
labels_map = {0 : 'T-Shirt', 1 : 'Trouser' , 2 : 'Pullover', 3 : 'Dress', 4 : 'Coat',5 : 'Sandal', 6 : 'Shirt', 7 : 'Sneaker' , 8 : 'Bag'

fig=plt.figure(figsize=(8,8))
columns=4;
rows=4;
```

```

for i in range(1,columns*rows+1):
    img_xy=np.random.randint(len(train_dataset));
    img=train_dataset[img_xy][0][0,:,:] #train_dataset을 이용한 3차원 배열 생성
    fig.add_subplot(rows,columns,i)
    plt.title(labels_map[train_dataset[img_xy][1]])
    plt.axis('off')
    plt.imshow(img,cmap='gray')
plt.show()

```



6. 심층 신경망 모델 생성

- ConvNet이 적용되지 않은 네트워크 생성

```

#심층 신경망 모델 생성
class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN,self).__init__()
        self.fc1=nn.Linear(in_features=784,out_features=256)
        self.drop=nn.Dropout(0.25)
        self.fc2=nn.Linear(in_features=256,out_features=128)
        self.fc3=nn.Linear(in_features=128,out_features=10)
    def forward(self,input_data):
        out=input_data.view(-1,784)
        out=F.relu(self.fc1(out))
        out=self.drop(out)
        out=F.relu(self.fc2(out))
        out=self.fc3(out)
        return out

```

- `def __init__(self)` : 클래스 형태의 모델은 `nn.Module`을 상속받고, `__init__()`은 객체가 속성 값을 초기화한다. `super()`를 통해 부모 클래스를 상속받는다.
- `nn.Linear(in_features=784,out_features=256)` : 입력의 크기와 출력의 크기를 파라미터값으로 받는다. >> 이때 입력의 크기만 넘겨주고 출력의 크기는 `forward()`의 연산결과가 된다.
- `nn.Dropout(p)` : p만큼 비율로 텐서의 값이 0이된다. 나머지는 기존값*(1/(1-p)) 만큼 곱해져 커진다.
- `forward()` : 순전파 학습을 진행 > 활성화 함수식에 입력값을 받아 예측값을 얻는 것
- `out=input_data.view(-1,784)` : `view`는 텐서의 크기를 변경해줌 / 여기서 -1은 첫 번째 차원은 파이토치가 알아서 하라는 것이며, 784는 두 번째 차원의 길이는 784로 하라는 것 >> 2차원 텐서로 변경하라.
- `out=F.relu(self.fc1(out))` : 활성화 함수를 지정하는 방법 중 하나

- `F.relu()` : `forward()` 함수에서 정의
- `nn.ReLU()` : `__init__()` 함수에서 정의

[nn.xx vs nn.functional.xx]

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

7. 심층 신경망에서 필요한 파라미터 정의

```
learning_rate=0.001;
model=FashionDNN();
model.to(device)

criterion=nn.CrossEntropyLoss(); #분류문제에 사용하는 손실 함수
optimizer=torch.optim.Adam(model.parameters(),lr=learning_rate);
print(model)
```

- 옵티마이저를 위한 경사 하강법은 Adam을 활용, 학습률은 0.001

8. 심층 신경망을 이용한 모델 학습

```
num_epochs=5
count=0

loss_list=[]
iteration_list=[]
accuracy_list=[]

predictions_list=[]
labels_list=[]

for epoch in range(num_epochs):
    for images,labels in train_loader:
        images,labels=images.to(device),labels.to(device)

        train=Variable(images.view(100,1,28,28))
        labels=Variable(labels)

        outputs=model(train) #학습 데이터를 모델에 적용
        loss=criterion(outputs,labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count+=1

    if not(count%50): # count를 50으로 나누었을 때 나머지가 0이 아니면 실행
        total=0
        correct=0;
        for images,labels in test_loader:
            images,labels=images.to(device),labels.to(device)
```



```

        labels_list.append(labels)
        test=Variable(images.view(100,1,28,28))
        outputs=model(test)
        predictions=torch.max(outputs,1)[1].to(device)
        predictions_list.append(predictions)
        correct+=(predictions==labels).sum()
        total+=len(labels)

    accuracy=correct*100/total
    loss_list.append(loss.data)
    iteration_list.append(count)
    accuracy_list.append(accuracy)

    if not(count%500):
        print("Iteration: {}, Loss: {}, Accuracy: {}%".format(count,loss.data,accuracy))

```

- 해당 반복마다 손실함수 loss와 정확도 accuracy를 출력한다.

```

Iteration: 500, Loss: 0.5898072719573975, Accuracy: 83.33999633789062%
Iteration: 1000, Loss: 0.4355780780315399, Accuracy: 84.13999938964844%
Iteration: 1500, Loss: 0.37107840180397034, Accuracy: 84.30999755859375%
Iteration: 2000, Loss: 0.3481672406196594, Accuracy: 85.29000091552734%
Iteration: 2500, Loss: 0.24756774306297302, Accuracy: 86.31999969482422%
Iteration: 3000, Loss: 0.29709920287132263, Accuracy: 86.58999633789062%

```

9. 합성곱 네트워크 생성

```

class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN,self).__init__()
        self.layer1=nn.Sequential(
            nn.Conv2d(in_channels=1,out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2,stride=2)
        )
        self.layer2=nn.Sequential(
            nn.Conv2d(in_channels=32,out_channels=64,kernel_size=3),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc1=nn.Linear(in_features=64*6*6,out_features=600)
        self.drop=nn.Dropout2d(0.25)
        self.fc2=nn.Linear(in_features=600,out_features=120)
        self.fc3=nn.Linear(in_features=120,out_features=10)

    def forward(self,x):
        out = self.layer1 (x)
        out = self.layer2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1 (out)
        out = self.drop(out)
        out = self.fc2(out)
        out = self.fc3(out)
        return out

```

- nn.Sequential() : 계층을 차례로 쌓을 수 있도록 손실함수와 같은 수식과 활성화 함수를 연결해주는 역할

- `nn.Conv2d(in_channels=1,out_channels=32, kernel_size=3, padding=1)` : 입력 채널의 개수, 출력 채널의 수, 커널의 크기, 패딩의 크기(출력 크기를 조정하기 위해 입력 데이터 주위를 0으로 채우는데 이때의 출력 크기를 의미)
- `nn.BatchNorm2d(32)` : 각 배치 단위별 평균과 분산을 이용하여 정규화하는 것을 의미
- `nn.MaxPool2d(2)` : 이미지 크기를 축소

Conv2d 계층에서의 출력 크기를 구하는 공식

$$\text{출력 크기} = (W - F + 2P) / S + 1$$

- W : 입력 데이터의 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

MaxPool2d 계층에서의 출력 크기를 구하는 공식

$$\text{출력 크기} = IF / F$$

- IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
- F : 커널 크기(kernel_size)

10. 파라미터 정의 및 모델 학습 및 성능 평가

```
# 파라미터 정의
learning_rate=0.001;
model=FashionCNN();
model.to(device)

criterion=nn.CrossEntropyLoss();
optimizer=torch.optim.Adam(model.parameters(),lr=learning_rate);
print(model)

# 모델 학습 및 성능 평가
num_epochs=5
count=0

loss_list=[]
iteration_list=[]
accuracy_list=[]

predictions_list=[]
labels_list=[]

for epoch in range(num_epochs):
    for images,labels in train_loader:
        images,labels=images.to(device),labels.to(device)

        train=Variable(images.view(100,1,28,28))
        labels=Variable(labels)

        outputs=model(train) #학습 데이터를 모델에 적용
        loss=criterion(outputs,labels)
        optimizer.zero_grad()
```

```

loss.backward()
optimizer.step()
count+=1

if not(count%50): # count를 50으로 나누었을 때 나머지가 0이 아니라면 실행
    total=0
    correct=0;
    for images,labels in test_loader:
        images,labels=images.to(device),labels.to(device)
        labels_list.append(labels)
        test=Variable(images.view(100,1,28,28))
        outputs=model(test)
        predictions=torch.max(outputs,1)[1].to(device)
        predictions_list.append(predictions)
        correct+=(predictions==labels).sum()
        total+=len(labels)

    accuracy=correct*100/total
    loss_list.append(loss.data)
    iteration_list.append(count)
    accuracy_list.append(accuracy)

if not(count%500):
    print("Iteration: {}, Loss: {}, Accuracy: {}".format(count,loss.data,accuracy))

```

```

Iteration: 500, Loss: 0.43928107619285583, Accuracy: 87.87000274658203%
Iteration: 1000, Loss: 0.3258974850177765, Accuracy: 88.0999984741211%
Iteration: 1500, Loss: 0.30115756392478943, Accuracy: 88.38999938964844%
Iteration: 2000, Loss: 0.1754811853170395, Accuracy: 89.62999725341797%
Iteration: 2500, Loss: 0.13775354623794556, Accuracy: 89.56999969482422%
Iteration: 3000, Loss: 0.24831737577915192, Accuracy: 88.94999694824219%

```

결론 : 심층 신경망과 비교하여 약간 더 정확도가 높음